

Operating system

[CSA0404]

CO2 AT1

Analgtical

Name: Sandhiya
Reg. NO: 192521224

1. CPU scheduling - waiting time and Turnaround time
Analysis (SJF - Non-Premptive)

Given

Process	Burst time (ms)
P1	10
P2	4
P3	6
P4	2

All Processes arrive at 0ms.

- a) Calculate waiting time and turnaround time

Step:- 1

Execution Order: $P4 \rightarrow P2 \rightarrow P3 \rightarrow P1$

Gantt chart

$| P4 |_2 | P2 |_6 | P3 |_{12} | P1 |_{22}$

Process	Burst time	ST	CT	WT	TAT
P ₄	2	0	2	0	2
P ₂	4	2	6	2	6
P ₃	6	6	12	6	12
P ₁	10	12	22	12	22

Average waiting time (also) second demand

$$= \frac{0+2+6+12}{4}$$

$$= 5 \text{ ms}$$

Average Turnaround time

$$= \frac{2+6+12+22}{4}$$

$$= 10.5 \text{ ms.}$$

Interpretation

- SJF executes the shortest Process first
- It minimizes average waiting time and turnaround time
- Short Process finish quickly, improving response time.

Round Robin scheduling (Time Quantum = 4 ms)

Process	Burst time
P1	20
P2	5
P3	13
P4	8

Time Quantum = 4ms

Deadlock - Safety Check using Banker's Algorithm.

Given:

Total resources

$$A = 10$$

$$B = 5$$

$$C = 7$$

Need matrix

Need = max Allocation

Process	Allocation	maximum	Need
P1	(0, 1, 0)	(7, 5, 3)	(7, 4, 3)
P2	(2, 0, 0)	(3, 2, 2)	(1, 2, 2)
P3	(3, 0, 2)	(9, 0, 2)	(6, 0, 0)
P4	(2, 1, 1)	(4, 2, 2)	(2, 1, 1)
P5	(0, 0, 2)	(5, 3, 3)	(5, 3, 1)

$$\begin{aligned} \text{Allocated} &= (7, 2, 5) \\ \text{Available} &= \text{Total} - \text{Allocated} \\ &= (3, 3, 2) \end{aligned}$$

Safety check

$$\text{Initial available} = (3, 3, 2)$$

- P2 Can execute $\rightarrow \text{AVL} = (5, 3, 2)$
 P4 Can execute $\rightarrow \text{AVL} = (7, 4, 3)$
 P5 Can execute $\rightarrow \text{AVL} = (7, 4, 5)$
 P1 Can execute $\rightarrow \text{AVL} = (7, 5, 5)$
 P3 Can execute $\rightarrow \text{AVL} = (10, 5, 7)$

The system is safe.

Safe sequence.

P2 $\rightarrow$ P4 $\rightarrow$ P5 $\rightarrow$ P1 $\rightarrow$ P3

Deadlock (RAG)

Detection

- Resource

Allocation

Graph

Given:

- P1 holds R1 and request R2.
 P2 holds R2 and request R3.
 P3 holds R3 and request R1.

Effect of increasing time quantum to 10ms.

$\Rightarrow$ Number of context switches decreases.

$\Rightarrow$ CPU needs less time switching between

Processes

$\Rightarrow$ CPU utilization improves.

$\Rightarrow$ Response time for short processes becomes

slightly slower.

CPU utilization multiprogramming Level Analysis.

Given

Probability of waiting for I/O, $P = 0.8$

CPU utilization formula.

$$u = 1 - P^n$$

n = Degree of multiprogramming.

a) Calculate CPU utilization ($n=4$).

$$\begin{aligned} u &= 1 - (0.8)^4 \\ &= 1 - 0.4096 \\ &= 0.5904. \end{aligned}$$

CPU utilization = 59.04%.

b) Calculate CPU utilization ($n=6$).

$$\begin{aligned} u &= 1 - (0.8)^6 \\ &= 1 - 0.2621 \\ &= 0.7379. \end{aligned}$$

CPU utilization = 73.79%.

2) Resource allocation graph.

$R_1 \rightarrow P_1$ (Allocated)
 $P_1 \rightarrow R_2$ (request)
 $R_2 \rightarrow P_2$ (allocated)
 $R_2 \rightarrow R_3$ (request)
 $R_3 \rightarrow P_3$ (allocated)
 $P_3 \rightarrow R_1$ (request).

Gantt Chart.

P_1 | P_2 | P_3 | P_4 | P_1 | P_2 | P_3 | P_4 | P_1 | P_3
 0 4 8 12 16 20 21 25 29 33 37

 P_3 | P_1
 43 46.

3) waiting Time and turnaround time

Process	BT	CT	WT	TAT
P1	20	46	26	46
P2	5	21	16	21
P3	13	42	29	42
P4	8	29	21	29

$$\text{Average waiting time} = \frac{26 + 16 + 29 + 21}{4} = 23 \text{ms.}$$

$$\text{Average Turnaround time} = \frac{46 + 21 + 42 + 29}{4} = 34.5.$$

c) Interpretation

=> increasing the degree of multiprogramming from

4 to 6 .

=> increases CPU utilization

=> more processors are available when one

Process waits for I/O .

=> Hence $n=6$ provides better CPU Performance

and efficiency .

Given

Buffer size = 6

initially

• empty = 6

mutex = 1

full = 0

$P1 \rightarrow P2 \rightarrow P3 \rightarrow C1 \rightarrow C2$

P = Producer, C = consumer

semaphore values After each operation

Operation	empty	full	mutex
initial	6	0	1
P1	5	1	1
P2	4	2	1
P3	3	3	1
C1	4	2	1
C2	5	1	1

b. How semaphore prevent race condition.
semaphores synchronize Producers and consumers.

• empty semaphore

Ensures a Producer inserts data only if there is free space in the buffer.

• full semaphore

Ensures a consumer removes data only if the buffer contains items.

• mutex semaphore

Allows only one process (Producer or consumer) to access the shared buffer at a time, preventing simultaneous modification.

If the mutex semaphore is removed, multiple processes or consumers can access the shared buffer simultaneously.

Justification:-

The mutex semaphore provides mutual exclusion. Without it, shared data is accessed concurrently without protection, making synchronization impossible, resulting in data inconsistency.

Process synchronization - critical section.

Given:

shared variable $x = 100$.

Process $P = P_1, P_2$.

a) A race condition occurs when two or more processes access and update the shared variable x at the same time without proper synchronization.

example:

$x = 100$

P_1 reads 100, Before P_1 updates it,
 P_2 also reads 100.

P_1 writes 101

P_2 also writes 101.

Although two increment operations were performed the value increased only once. This is called a lost update. When this happens repeatedly, the final value becomes incorrect.

b)

$x = 100$.

Process P_1 increment x 1000 times

Process P_2 increment x 1000 times.

Total increment = $1000 + 1000$
= 2000.

Expected value of $x = 100 + 2000$
= 2100.

$x = 2100$.

Hardware-based solution.

Test-and-set (TSL) instruction.

→ it is an atomic hardware instruction that allows only one process to enter the critical section at a time, preventing race conditions.

Software-based solution

Semaphore

→ A semaphore controls access to the critical section using `wait()` and `signal()` operations, ensuring that only one process accesses the shared variable at a time.

Thread scheduling and Performance.

Given

No. of user-level thread = 40

CPU cores = 8

CPU time

per thread = 12 ms

I/O time = negligible.

a) Many-to-one

→ Many user threads are mapped to a single kernel thread.

one-to-one

→ Each user thread is mapped to a separate kernel thread.

only one thread can execute at a time.

Does not utilize multiple processors effectively

Lower overhead but poor performance for parallel tasks.

multiple threads can execute simultaneously on multiple CPU cores.

Fully utilizes multiple processors.

Higher overhead but better performance and responsiveness.

Many to one

=> only one kernel thread executes all users threads

=> Only 1 out of 8 CPU cores is used.

=> Estimated CPU utilization of ~~100%~~ 12.5%.

One to one

=> Each thread has its own kernel thread.

=> up to 8 threads can run simultaneously on 8 CPU cores.

=> Estimated CPU utilization of 100%.

The one-to-one multithreading model is the best choice.

Justification

=> it allows multiple threads to execute in parallel on all 8 cores.

=> it provides higher CPU utilisation better Performance, faster execution.

9.

Monitor Synchronization.

a) Mutual Execution

A Monitor allow only one process to access the shared printer at a time, other processes wait until the printer become available, Preventing race condition.

b) Role of wait() and signal()

- wait() suspends a process until the printer is free.

- signal() wakes up a waiting process after the current job is completed.

c) If signal() is omitted.

Waiting process remain blocked indefinitely, causing starvation and reducing system Performance.

Deadlock Prevention Vs Avoidance.

a) Deadlock Prevention.

Deadlock Prevention avoids deadlock by eliminating one of the four necessary conditions. For example, removing the hold and wait condition by requiring a process to request all required resources at once.

b) Prevention Vs Avoidance.

Prevention: Forces processes to request all resources before execution.

Avoidance: Grants resources only if the system remains in a safe state.

c) Comparison.

Prevention	Avoidance.
→ Prevents deadlock by restricting resource allocation. → Lower resource utilization. → simple but less efficient.	→ prevents deadlock checking the safe state → better resource utilization. → more efficient but more complex.